WEEK 1 · LESSON 1 OF 24 · HANDOUT

What Deep Learning Is — and Your First Neuron

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=aircAruvnKk — open the lesson online to watch inside the course.

🎓 **Go deeper:** MIT 6.S191: Introduction to Deep Learning (2024) — https://www.youtube.com/watch?v=ErnWZxJovaM

🎓 **Go deeper:** Harvard CS50 AI, Lecture 5: Neural Networks — https://www.youtube.com/watch?v=mFZazxxCKbw

Your goal in this lesson: Explain what deep learning is and where it is used, describe an artificial neuron in plain words, and compute one — and a whole layer of them — with NumPy.

From machine learning to deep learning

In **Machine Learning Fundamentals** you gave models features that people had prepared: income, loan size, size in m². That works well for tables. But what are the "features" of a photo of a cassava leaf, a voice saying "Muraho", or a chest X-ray? Nobody can write them down by hand.

Deep learning solves this. A **neural network** with many layers learns the features **by itself**, straight from raw pixels, sound waves or text. The first layers learn simple patterns (edges, short sounds), the next layers combine them into bigger ones (leaf spots, syllables), and the last layers make the decision.

	Classic machine learning	Deep learning
Features	Designed by people	Learned from raw data
Best for	Tables of numbers	Images, sound, text, video
Data needed	Hundreds to thousands of rows	Usually thousands to millions of examples
Computing	A laptop	Often a GPU (Colab gives one free)

Where deep learning is used

- **Health:** reading X-rays and eye scans, digital stethoscopes that recognise heart sounds.
- **Farming:** phone apps that recognise crop diseases from a photo of a leaf.
- **Language:** speech recognition, translation and chatbots — including for Kinyarwanda, Swahili and other African languages.
- **Everyday tools:** face unlock, photo search, keyboard suggestions, fraud detection.

Large language models such as ChatGPT and Claude are deep neural networks too — you will meet their key idea, **attention**, in Week 4.

The artificial neuron

A neuron is a tiny calculator:

1. It takes some **inputs** (x_1, x_2, …).

2. It multiplies each input by a **weight** (w_1, w_2, …) — how important that input is.
3. It adds everything up, plus a **bias** b.
4. It passes the total through an **activation function**, which decides how strongly the neuron "fires".

> *output = activation($w_1x_1 + w_2x_2 + \ldots + b$)*

The weights and bias are what the network **learns**. Everything else is fixed.

One neuron in NumPy

Suppose a neuron decides whether you should take a moto instead of walking. Inputs: distance in km, whether it is raining (0/1), and how late you are in minutes.

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

x = np.array([3.0, 1.0, 10.0])          # 3 km, raining, 10 minutes late
w = np.array([0.8, 1.5, 0.2])           # importance of each input
b = -4.0                                # bias: a starting preference for walking

z = np.dot(w, x) + b                    # weighted sum
print("weighted sum:", round(z, 2))
print("probability of taking a moto:", round(sigmoid(z), 3))
```

The weighted sum is 0.8 × 3 + 1.5 × 1 + 0.2 × 10 − 4 = **1.9**, and the sigmoid turns it into a probability of about **0.87**. Change the inputs and see how the output moves.

Many neurons = a layer

A **layer** is many neurons that all look at the same inputs, each with its own weights. With NumPy, a whole layer is one **matrix multiplication** — the matrix maths you learned in Math & Python for AI:

```
rng = np.random.default_rng(0)
X = rng.random((5, 3))          # 5 examples, 3 inputs each
W = rng.normal(0, 1, (3, 4))    # 3 inputs -> 4 neurons
b = np.zeros(4)

layer_output = sigmoid(X @ W + b)
print(layer_output.shape)       # (5, 4): 5 examples, 4 neuron outputs each
print(layer_output.round(2))
```

`X @ W` computes all 20 weighted sums (5 examples × 4 neurons) at once. This is why deep learning runs so well on **GPUs**, chips built to do huge matrix multiplications in parallel.

Stacking layers: a deep network

A network passes the output of one layer as the input to the next:

> *inputs → hidden layer 1 → hidden layer 2 → … → output layer*

"Deep" simply means **many hidden layers**. Modern image models have dozens; large language models have around a hundred, with billions of weights.

```
W1, b1 = rng.normal(0, 1, (3, 4)), np.zeros(4)   # layer 1: 3 -> 4
W2, b2 = rng.normal(0, 1, (4, 1)), np.zeros(1)   # layer 2: 4 -> 1

hidden = sigmoid(X @ W1 + b1)
output = sigmoid(hidden @ W2 + b2)
print(output.round(3).ravel())
print("number of learnable numbers:", W1.size + b1.size + W2.size + b2.size)
```

These random weights give meaningless answers. **Training** — next lessons — finds weights that give good ones.

Key terms

- **Deep learning** — machine learning with neural networks of many layers that learn features from raw data.
- **Neuron** — weighted sum of inputs plus a bias, passed through an activation function.
- **Weight / bias** — the numbers a network learns.
- **Layer** — a group of neurons that see the same inputs.
- **Hidden layer** — a layer between the input and the output.
- **Parameters** — all the weights and biases of a network.

Remember this: A neural network is layers of simple neurons; each layer is a matrix multiplication followed by an activation, and learning means finding good weights.

Name: _____ Date: _____

Activity

Task 1

In the moto neuron, find an input where the probability is below 0.5. Which input matters most?

Task 2

Change the bias to −8. What happens, and what does the bias mean in plain words?

Task 3

Build a layer with 3 inputs and 10 neurons for 7 examples. What shape is the output?

Task 4

Count the parameters of a network with layers 64 → 32 → 10. (Hint: weights plus biases for each layer.)

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is the main difference between deep learning and classic machine learning?
   - A. Classic ML uses GPUs
   - B. Deep learning uses no data
   - C. Deep learning learns features from raw data with many layers
   - D. Deep learning only works on tables
2. What does a neuron compute before its activation function?
   - A. A random number
   - B. The average of the outputs
   - C. A sort of the inputs
   - D. A weighted sum of its inputs plus a bias
3. What does a network learn during training?
   - A. The weights and biases
   - B. The number of examples
   - C. The inputs
   - D. The activation formula
4. How is a whole layer of neurons computed efficiently?
   - A. With a loop over each neuron
   - B. With sorting
   - C. With one matrix multiplication
   - D. With a pie chart
5. What does 'deep' mean in deep learning?
   - A. The data is very large
   - B. It uses deep colours
   - C. The network has many hidden layers
   - D. It runs underground
6. A layer has 3 inputs and 4 neurons. How many weights does it have (without biases)?
   - A. 3
   - B. 12
   - C. 4
   - D. 7

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.